CyberRange OS

AI-Augmented Red Team / Blue Team Training Platform

Self-hosted, locally-inferenced, accreditation-ready

Department Cybersecurity Lab
NBA/NAAC-aligned · Locally hosted LLMs

Document	Project Design & Implementation Report
Scope	Requirements, Architecture, Implementation, Deployment
Backend	Go 1.25 + Fiber
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind
Local LLM	Ollama / vLLM / TGI (open-weight models)
Repository	`github.com/MD-HACKER07/CyberRange-OS`

Every LLM call goes to a private endpoint; no student data, lab logs, or exploit attempts leave the institution's network.

September 1, 2026

Contents

1 Executive Summary

CyberRange OS is a self-hosted web platform that lets cybersecurity students practice, under supervision, both **offensive** security (reconnaissance, exploitation, reporting against an isolated, intentionally-vulnerable target range) and **defensive** security (log analysis, threat detection, incident response) through a real SOC-style workflow. Every exercise is assisted by a **locally-hosted, open-weight LLM**, so no student data, lab logs, or exploit attempts ever leave the institution's network.

The platform produces structured evidence — attempt logs, scores, rubric-graded reports, and time-to-detect metrics — that maps to Course Outcomes (CO) and Programme Outcomes (PO) for **NBA** accreditation, and aggregates research/innovation evidence for **NAAC**.

1.1 Roles Served

- **Student** — enroll in batches, run Red/Blue exercises, submit reports, view CO/PO progress and leaderboard rank.
- **Faculty/Instructor** — author exercises with rubrics, map to CO/PO, grade reports (override AI suggestions), view class dashboards.
- **Lab Admin** — provision range targets, manage the LLM model registry and prompts, manage users/RBAC, view the full audit log.
- **Auditor (read-only)** — view aggregated CO/PO/NAAC evidence exports only.

2 Capabilities — What the Platform Can Do

3 System Architecture

CyberRange OS is a service-oriented, on-prem monorepo. The Go API is a modular monolith — one deployable binary containing the auth, orchestrator, LLM gateway, log-ingest, MITRE, analytics, reporting, and AI-security subsystems — each behind a clean interface so it can later be split into its own process.

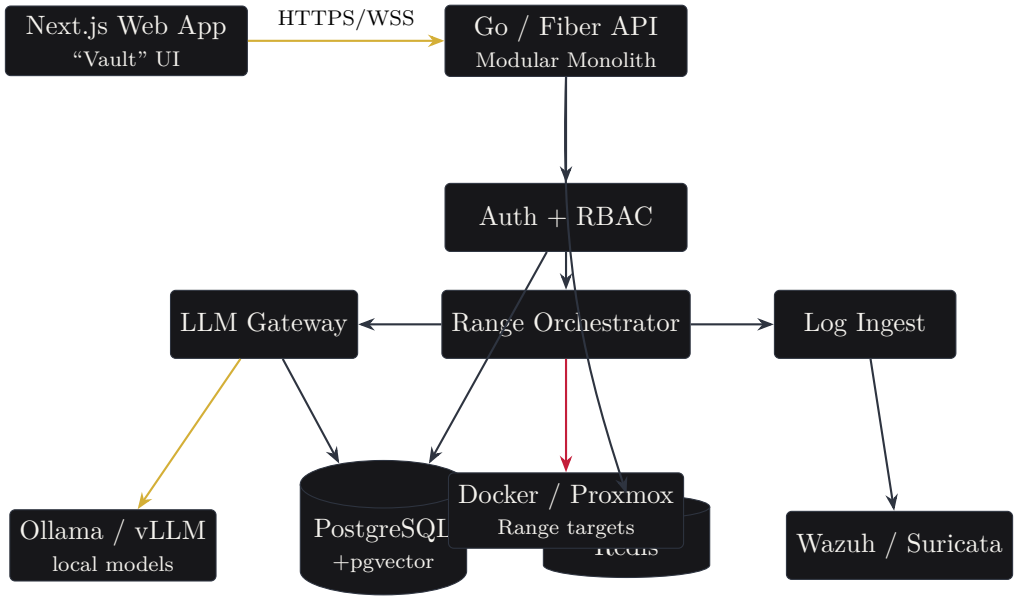

Figure 1: High-level architecture. Gold = brand/LLM paths, red = Red Team range paths.

Module	Capability
Auth & Identity	JWT access + rotating refresh tokens, institution SSO (OIDC) with local fallback, four RBAC roles, append-only audit trail.
Red Team Range	Per-session isolated range provisioning, browser-driven command execution in a Kali container, full command logging, LLM pentest copilot (suggest → approve → execute), MITRE auto-tagging, live resource stats, tool quick-actions.
LLM Gateway	Local-inference-only guard, model registry, versioned system prompts, streaming, per-session token budgets, complete prompt/completion logging for reproducibility.
Blue Team SOC	Wazuh + Suricata ingestion into a common alert schema, live alert console with severity filters, SOC copilot (summary/verdict/next-step), MTTD/MTTR timers, playbooks, faculty ground-truth labeling, accuracy dashboard.
MITRE ATT&CK Engine	STIX/curated dataset ingest, pgvector semantic search, LLM disambiguation, per-session technique tracker.
Reporting Engine	Markdown editor with live preview, AI grading assistant (faculty authoritative), PDF export, combined portfolio PDF.
Accreditation Analytics	CO/PO matrix, weighted direct/indirect attainment, heatmap, NBA CSV & NAAC evidence exports, course-exit surveys.
AI Security	PyRIT/Garak (or built-in probe battery) run against the institution's own local model; results dashboard.
Gamification	XP weighted by difficulty and inverse copilot reliance; red/blue/combined leaderboards; pluggable external CTF feed.
Voice Assistant	Real-time speech Q&A about department routine and platform usage, answered by the local model, with an admin-editable knowledge base.
Admin & Observability	Range target provisioning, LLM registry, RBAC, audit viewer with CSV export, system health, Prometheus metrics.

Table 1: Functional capability matrix.

3.1 Backend Packages (`apps/api/internal`)

4 Network Isolation & Safety Model

The range is structurally prevented from touching arbitrary hosts: attack targets are selected only from a pre-registered dropdown of range assets — no free-text host field exists anywhere. Range networks are provisioned with the Docker `internal:true` flag, so Docker attaches *no gateway to the WAN*.

4.1 Non-Negotiable Guardrails

1. Targets come only from the `range_targets` registry (dropdown), never free text.
2. The copilot never auto-executes; a human clicks **Approve & Run**, and every approved action is logged with student, target, timestamp, and full command.
3. All LLM traffic goes to `LLM_BASE_URL`; the gateway refuses to start if that resolves to a public IP (unless explicitly overridden for a lab-controlled network).
4. The audit log is append-only, enforced by a database trigger.

5 Red Team Workflow

The primary evidence artifact is the `session_command_log` table: every executed command, its captured output, exit code, whether it was AI-suggested, and the mapped ATT&CK technique. Copilot *suggestions* are stored even when never approved, proving human-in-the-loop control.

Package	Responsibility
`config`	Env-driven configuration; every dependency is a configurable URL.
`auth`	JWT access/refresh (rotating), OIDC SSO, local password, RBAC middleware.
`audit`	Append-only audit trail (DB trigger blocks UPDATE/DELETE).
`llm`	Local-inference gateway: egress guard, model registry, versioned prompts, streaming, budgets, call loggin
`orchestrator`	`RangeProvisioner` interface + `DockerDriver` and `ProxmoxDriver`.
`ingest`	Polls Wazuh REST + tails Suricata EVE JSON into the common Alert schema.
`mitre`	ATT&CK dataset, pgvector semantic search, LLM auto-tagging.
`report`	Markdown → HTML → PDF rendering.
`aisec`	PyRIT/Garak runner against the local model (built-in probe fallback).
`store`	Data access for courses, batches, exercises, sessions, reports, analytics.
`server`	Fiber wiring + per-module HTTP/WS handlers.

Table 2: Backend module responsibilities.

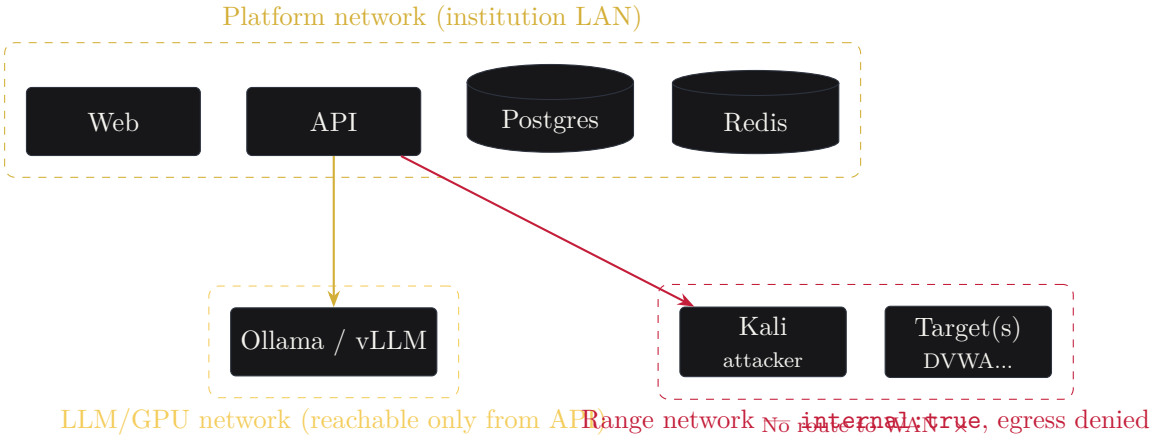

Figure 2: Three logical networks. Range traffic cannot reach the internet.

6 Blue Team Workflow

MTTD (mean time to detect) starts when the paired Red Team action occurs; MTTR (mean time to respond) stops when the student marks the alert resolved with a written response. Faculty set the ground-truth label per alert, so the platform measures both **copilot accuracy** and **student accuracy** against ground truth (a CyberSOCEval-style comparison).

7 Core Data Model

Additional tables include `llm_prompts` (versioned), `llm_calls` (full input/output logging), `copilot_suggestions` `playbooks`, `ai_redteam_scans`, `leaderboard_entries`, `surveys`, and `audit_log`.

8 Technology Stack

9 Local LLM Gateway — "No Data Leaves" Guarantee

All modules call through a single gateway; nothing in the frontend talks to the model directly. The gateway enforces the local-inference guarantee at startup:

```
func AssertLocalEndpoint(rawURL string, allowPublic bool) (*EgressAssertion, error) {
    // resolve host -> IPs; every IP must be loopback/private (RFC1918, CGNAT).
    // If any resolves to a public address and the override is off, refuse to start.
}
```

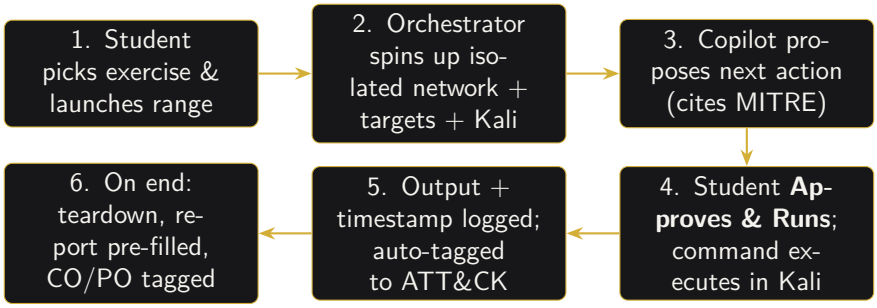

Figure 3: Human-in-the-loop Red Team session lifecycle.

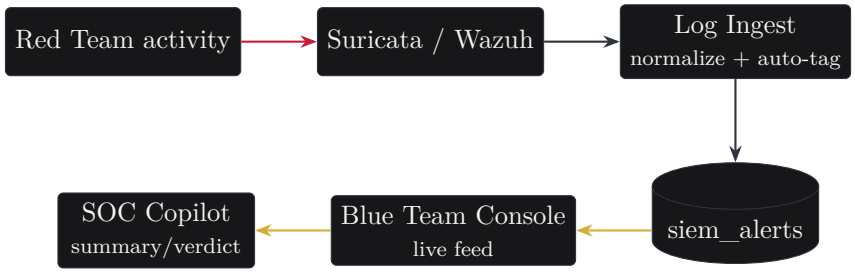

Figure 4: Real telemetry pipeline: attacks generate IDS events that students triage live.

Listing 1: Egress guard (simplified).

Per-module system prompts are stored in the database and versioned; every call records the model and prompt version used, so graded work stays reproducible for accreditation review. Modules: `pentest-copilot`, `soc-copilot`, `report-grading`, `mitre-tagging`, `red-teaming-target`, and `assistant`.

10 Real-Time Voice Department Assistant

Students and faculty can ask — by voice or text — about department routine, timings, contacts, rules, or how to use the platform. Speech-to-text and text-to-speech run in the browser (Web Speech API); the *answer* is generated by the local model, so knowledge stays on-prem. Faculty/admin edit a knowledge base that grounds the answers; the assistant is instructed not to invent facts it does not have.

11 Accreditation Analytics (CO/PO, NBA/NAAC)

Each graded exercise contributes an attainment data point per tagged CO. The platform computes the standard weighted direct/indirect attainment (configurable, e.g. 80/20), rolls it up to POs via the mapping matrix, and renders a heatmap. Exports include an NBA-format attainment CSV per batch and a NAAC evidence summary (student projects, AI-security findings, range usage).

$$CO_{final} = w_d \cdot Direct\% + w_i \cdot Indirect\%, \qquad PO_{score} = \frac{\sum_i CO_i \cdot weight_i}{\sum_i weight_i}$$

12 Deployment

```
git clone git@github.com:MD-HACKER07/CyberRange-OS.git && cd CyberRange-OS
cp .env.deploy.example .env.deploy   # set POSTGRES_PASSWORD, JWT_SECRET, admin pass
```

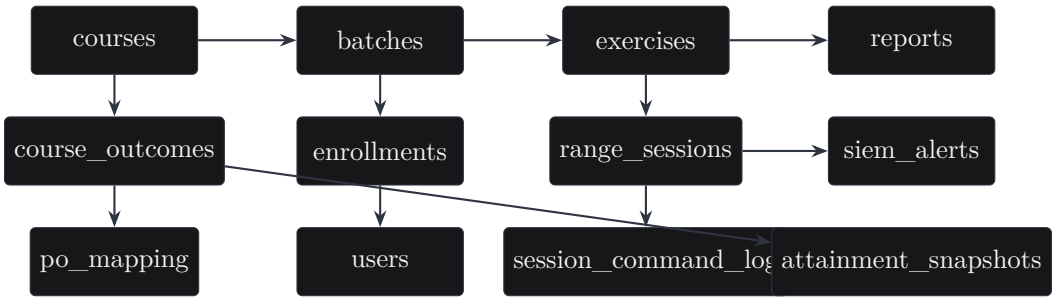

Figure 5: Selected core tables and relationships (full schema in `0001_init.sql`).

Layer	Technology
Backend API	Go 1.25 + Fiber v2
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS
Realtime	WebSockets via Redis pub/sub fan-out
Primary DB	PostgreSQL 16 with pgvector
Cache / Queue / Pub-Sub	Redis 7
Auth	JWT (access + rotating refresh), OIDC SSO, local fallback
Local LLM runtime	Ollama (dev) / vLLM (prod) — open-weight models
Range orchestration	Docker Engine API (`DockerDriver`); Proxmox driver stub
SIEM / IDS	Wazuh manager + Suricata (EVE JSON)
Offensive tooling	Kali container: nmap, Metasploit, sqlmap, gobuster, nikto, hydra
AI red-teaming	PyRIT / Garak against the local model
Reporting	Headless Chromium / wkhtmltopdf for PDF
Observability	Prometheus metrics + structured JSON logging (zerolog)

Table 3: Technology choices per layer.

```
docker build -t cyberrange/kali-attacker:latest infra/kali
docker compose -f infra/docker-compose.deploy.yml --env-file .env.deploy up -d --build
```

Listing 2: One-command server bring-up.

13 Implementation Status — What We Built

13.1 Verification

The Go backend compiles cleanly and passes unit tests (including the egress-guard suite). The Next.js frontend type-checks and builds all 17 routes. The full stack has been run end-to-end: database migrations and reference-data seeding, admin login, and a live connection to a local Ollama model serving `llama3.2:3b` and `nomic-embed-text`.

14 How Students & Faculty Use It

1. **Log in** with college email/roll number (SSO or local).
2. **Dashboard**: batch, active session card, CO/PO progress, leaderboard snippet.
3. **Red Team**: choose a published exercise, launch the range, run recon/exploit commands, approve copilot suggestions; every action is logged and MITRE-tagged.
4. **Blue Team**: triage live alerts with the SOC copilot, resolve with a written note; MTTD/MTTR tracked.

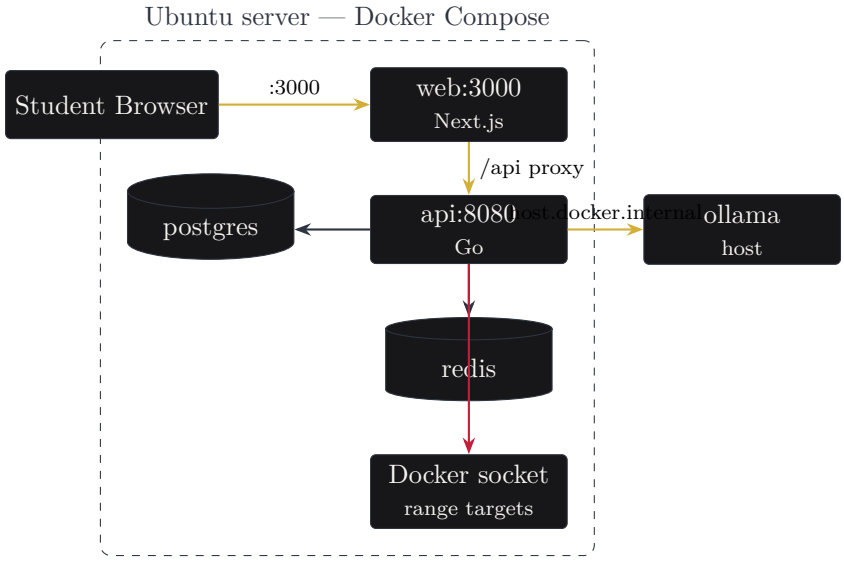

Figure 6: Single-server deployment (`docker-compose.deploy.yml`). Native Ollama reused via the private Docker bridge.

5. **Reports**: write and submit a pentest/incident report (auto-seeded from the session timeline), export PDF, build a portfolio.
6. **Assistant / Learning Paths / Leaderboard**: ask questions by voice, track certification coverage and XP.
7. **Faculty**: author exercises, grade with rubric (override AI), set alert ground truth, view CO/PO heatmap and SOC accuracy, export NBA/NAAC evidence.

15 Roadmap / What We Can Do Next

- Complete the **Proxmox driver** for full VM-level isolation in production.
- Add **browser desktop** access (Apache Guacamole) for GUI tools like Burp Suite.
- Offline **Whisper** service for fully-local speech-to-text.
- TLS on the Docker Engine API and mutual-TLS between services.
- Deeper **CyberSOCEval** analytics and per-technique attainment.
- Pluggable external **CTF** leaderboard ingestion.
- Grafana dashboards for GPU utilization and range host health.

Appendix A — Representative API Surface

```
POST   /api/auth/login | /refresh | /logout        GET /api/me
CRUD   /api/courses, /api/batches, /api/exercises
POST   /api/range-sessions          POST /api/range-sessions/:id/end
WS     /ws/range-sessions/:id/terminal
POST   /api/range-sessions/:id/copilot/suggest | /copilot/execute | /exec
GET    /api/range-sessions/:id/stats | /commands | /techniques
GET    /api/siem/alerts        POST /api/siem/alerts/:id/copilot/summarize
POST   /api/siem/alerts/:id/resolve | /ground-truth | /detect
WS     /ws/siem/alerts/live    GET /api/siem/accuracy | /metrics
```

Component	Status	Notes
Auth, RBAC, audit trail	**Done**	JWT rotation, OIDC, 4 roles, append-only log.
Courses/batches/exercises CRUD	**Done**	Faculty authoring, publish workflow.
Range orchestrator (Docker)	**Done**	Isolated networks, exec, teardown, stats.
Proxmox driver	**Interface stub**	Same interface, ready for completion.
LLM Gateway + egress guard	**Done**	Model registry, versioned prompts, budgets, logging.
Red Team console	**Done**	Live terminal, copilot approve/run, tool palette, ATT&CK tracker.
Blue Team console	**Done**	Alert feed, SOC copilot, MTTD/MTTR, filters, ground-truth.
Log ingest (Wazuh + Suricata)	**Done**	Common schema, auto-tag, live stream.
MITRE engine	**Done**	Curated + STIX ingest, pgvector semantic search.
Reporting + PDF	**Done**	Editor, AI grade, portfolio export.
Accreditation analytics	**Done**	CO/PO attainment, heatmap, NBA/NAAC exports, surveys.
AI Security (PyRIT/Garak)	**Done**	CLI integration + built-in probe fallback.
Gamification & leaderboards	**Done**	XP with assistance ratio, tracks.
Voice assistant	**Done**	Local-LLM Q&A + editable knowledge base.
Demo content seed	**Done**	Course, faculty+student, 3 exercises, playbook.
Docker Compose deployment	**Done**	Single-server compose + Suricata profile.

Table 4: Build status across all phases (1–10 plus enhancements).

```
CRUD   /api/reports    POST /api/reports/:id/submit | /grade | /ai-grade
GET    /api/reports/:id/pdf    GET /api/reports/portfolio/:uid
GET    /api/attainment/batch/:bid | /batch/:bid/po | /export/:bid
GET    /api/leaderboard/:batch_id    GET /api/attack/techniques
POST   /api/assistant/ask     GET/PUT /api/assistant/knowledge
CRUD   /api/admin/llm-models  /api/admin/llm-prompts
POST   /api/admin/range-targets    GET /api/admin/audit-log | /health
POST   /api/ai-security/scan   GET /api/ai-security/results
```

End of document — CyberRange OS Project Report.